

Assignment

Natural Language E-commerce Database Agent

Using LangChain Agents, Custom Tools, and SQLite

Target Audience	Python, SQL, API, and introductory LangChain learners
Difficulty Level	Intermediate to Advanced
Estimated Duration	4 to 6 hours
Primary Tools	Python, LangChain, SQLite, LLM, Custom Tool Calling
Final Objective	Answer natural language business queries by connecting an AI agent to an SQLite e-commerce database

Table of Contents

1. 1. Assignment Overview
2. 2. Business Scenario
3. 3. Required Outcome
4. 4. Functional Requirements
5. 5. Database Schema
6. 6. Agent and Tool Requirements
7. 7. Security and Safety Requirements
8. 8. Version-Specific LangChain Requirements
9. 9. Recommended Project Structure
10. 10. Application Flow
11. 11. Prompting Requirements
12. 12. Sample Queries
13. 13. Non-Functional Requirements
14. 14. Optional Advanced Features
15. 15. Expected Deliverables
16. 16. README Requirements
17. 17. Evaluation Rubric
18. 18. Final Submission Checklist

1. Assignment Overview

The objective of this assignment is to build an AI agent that can answer business questions from an e-commerce SQLite database using natural language. The user should be able to ask plain-English questions, and the agent should use a database tool to retrieve accurate answers from the SQLite database.

Participants must demonstrate how an LLM-powered agent can reason over a business question, decide whether database access is needed, call a custom tool, execute a safe SQL query, and return a clear, business-friendly response.

- Build an agent using LangChain.
- Create and seed a realistic SQLite e-commerce database.
- Implement a custom database query tool.
- Restrict the tool to read-only SELECT queries.
- Support natural language questions from business users.
- Demonstrate implementation awareness for LangChain versions below 1.0 and version 1.0 or above.

2. Business Scenario

You are working as an AI engineer for an e-commerce company. The business team frequently asks data-related questions, but they do not know SQL. They usually depend on developers or data analysts to extract information from the database.

The company wants to build a prototype of an AI-powered database assistant that allows business users to ask questions in plain English and receive accurate answers from the e-commerce database.

The prototype should use Python, LangChain, SQLite, an LLM with tool-calling capability, a custom database tool, and a simple natural language interface.

3. Required Outcome

19. A user enters a natural language business query.
20. The LangChain agent receives the query.
21. The agent decides whether database access is required.
22. The agent calls a custom SQLite database tool.
23. The tool validates and executes a safe SQL query.
24. The tool returns the result to the agent.
25. The agent converts the result into a clean business answer.
26. The final response is shown to the user.

Example Expected User Experience

User: Which are the top 3 customers by total purchase amount?

Agent: The top 3 customers by total purchase amount are:

1. Rohan Mehta - Rs. 42,500
2. Priya Sharma - Rs. 38,900
3. Neha Gupta - Rs. 35,700

These customers have generated the highest total revenue based on completed orders.

4. Functional Requirements

4.1 Database Creation

Participants must create a local SQLite database named ecommerce.db. The database should be placed in a data folder and created through scripts so that the evaluator can reproduce the database from scratch.

4.2 Minimum Sample Data

Table	Minimum Records
customers	10
products	15
orders	25
order_items	40

The data should include customers from different cities, products from multiple categories, completed/pending/cancelled orders, products with low stock, multiple orders per customer, and multiple products per order.

5. Database Schema

5.1 customers

Column	Type	Description
customer_id	INTEGER PRIMARY KEY	Unique customer ID
name	TEXT	Customer name
email	TEXT	Customer email
city	TEXT	Customer city
signup_date	TEXT	Date when customer joined

5.2 products

Column	Type	Description
product_id	INTEGER PRIMARY KEY	Unique product ID
name	TEXT	Product name
category	TEXT	Product category
price	REAL	Product price
stock_quantity	INTEGER	Available stock

5.3 orders

Column	Type	Description
--------	------	-------------

order_id	INTEGER PRIMARY KEY	Unique order ID
customer_id	INTEGER	Customer who placed the order
order_date	TEXT	Date of order
status	TEXT	completed, pending, or cancelled
total_amount	REAL	Total order amount

5.4 order_items

Column	Type	Description
order_item_id	INTEGER PRIMARY KEY	Unique order item ID
order_id	INTEGER	Related order
product_id	INTEGER	Related product
quantity	INTEGER	Quantity purchased
unit_price	REAL	Product price at time of order

6. Agent and Tool Requirements

6.1 Agent Requirement

Participants must build an agent that can answer user questions by interacting with the SQLite database through a tool. The agent should not expose raw SQL to the user unless the user explicitly asks for it.

- Understand natural language business questions.
- Select the database tool when data is required.
- Generate SQL using the known database schema.
- Interpret raw database results.
- Return concise and business-friendly answers.
- Avoid hallucinating numbers or unsupported conclusions.

6.2 Custom Tool Requirement

Participants must create a custom LangChain tool that connects to the SQLite database. The tool should be reusable and should not be hardcoded for a single question.

Tool Element	Expected Value
Suggested tool name	query_ecommerce_database
Input	A SQL SELECT query or structured database request
Output	Query result or a graceful error message
Responsibility	Connect to ecommerce.db, validate query safety, execute query, and return results
Restriction	Only SELECT queries are allowed

Suggested tool description:

Use this tool to query the SQLite ecommerce database. The database contains customers, products, orders, and order_items tables. Only use SELECT queries. Do not perform INSERT, UPDATE, DELETE, DROP, ALTER, or TRUNCATE operations.

7. Security and Safety Requirements

This assignment must include database safety controls. The purpose is to simulate responsible agent design for enterprise applications.

7.1 Allow Only SELECT Queries

The database tool must reject unsafe SQL statements such as INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, CREATE, or REPLACE.

7.2 Prevent Multiple Statements

The tool should reject multiple SQL statements separated by semicolons. For example, the following query must be rejected:

```
SELECT * FROM customers; DROP TABLE orders;
```

7.3 Limit Output Size

The tool should limit query results to a reasonable number of rows. If the user asks for many records, the agent should summarize the result.

7.4 Handle Errors Gracefully

The application should not crash because of an invalid query, missing database file, or database connection issue. It should return a clear and actionable error message.

8. Version-Specific LangChain Requirements

Participants must understand that LangChain has evolved significantly. This assignment should be attempted with either one implementation style or both implementation styles, depending on training scope.

8.1 Option A: LangChain Version Less Than 1.0

This version demonstrates legacy LangChain agent development patterns. Participants may use older APIs such as `initialize_agent`, `AgentExecutor`, `Tool`, `StructuredTool`, or `AgentType` based on the compatible version selected.

- Create a custom SQLite query tool.
- Register the tool with the agent.
- Pass user queries to the agent.
- Return final natural language answers.
- Clearly mention in the README that this is the legacy implementation.

Suggested file: requirements-legacy.txt

```
langchain<1.0
langchain-community
```

```

langchain-openai
python-dotenv

# If using Groq:
langchain-groq
groq

```

8.2 Option B: LangChain Version Greater Than or Equal to 1.0

This version demonstrates modern LangChain agent development. Participants should use the modern `create_agent` style with tool definitions, a system prompt, and a tool-calling capable model.

- Use `create_agent` or the current recommended agent creation API.
- Define the database tool clearly.
- Provide the database schema in the system prompt.
- Invoke the agent with the user message.
- Return a final business-friendly answer.

Suggested file: `requirements-modern.txt`

```

langchain>=1.0
langchain-community
langchain-openai
python-dotenv

# If using Groq:
langchain-groq
groq

```

9. Recommended Project Structure

```

ecommerce-agent-assignment/
|
|-- README.md
|-- .env.example
|-- requirements-legacy.txt
|-- requirements-modern.txt
|
|-- data/
|   |-- ecommerce.db
|
|-- scripts/
|   |-- create_database.py
|   |-- seed_database.py
|
|-- src/
|   |-- db/
|   |   |-- connection.py
|   |   |-- schema_description.py
|   |
|   |-- tools/
|   |   |-- ecommerce_sql_tool.py
|   |
|   |-- agents/
|   |   |-- legacy_agent.py
|   |   |-- modern_agent.py
|

```

```

| |
| | -- prompts/
| | | -- system_prompt.py
| |
| | -- app.py
|
|-- tests/
|   |-- test_database.py
|   |-- test_sql_tool.py
|   |-- test_agent_queries.py
|
|-- sample_queries.md

```

10. Application Flow

```

User enters natural language query
      |
      v
Application sends query to LangChain agent
      |
      v
Agent understands the user intent
      |
      v
Agent decides whether database access is required
      |
      v
Agent calls ecommerce SQLite database tool
      |
      v
Tool validates and executes SELECT query
      |
      v
Tool returns database result
      |
      v
Agent converts result into business-friendly response
      |
      v
User receives final answer

```

11. Prompting Requirements

Participants must create a strong system prompt that defines the agent role, available database tables, safety restrictions, and expected response style.

You are an AI assistant for an e-commerce business.

You can answer business questions using the ecommerce SQLite database.

The database has the following tables:

- customers
- products
- orders
- order_items

Use the database tool only when the user asks a question that requires data.

Important rules:

- Only generate SELECT queries.
- Never generate INSERT, UPDATE, DELETE, DROP, ALTER, or TRUNCATE queries.
- Always explain results in simple business language.
- If the data is unavailable, clearly say that the answer cannot be determined from the database.
- Do not make up numbers.
- If a query returns no records, explain that no matching data was found.

12. Sample Queries to Support

27. What is the total revenue from completed orders?
28. Which customer has spent the most money?
29. Show the top 5 products by quantity sold.
30. Which product category generated the highest revenue?
31. How many orders are pending?
32. Show all cancelled orders with customer names.
33. Which customers are from Delhi?
34. Which products have stock below 10?
35. What is the average order value?
36. Show total revenue by month.
37. Which city has the highest number of customers?
38. Which customers have placed more than 2 orders?
39. What are the top 3 most expensive products?
40. Which product has never been ordered?
41. Show total sales by product category.

13. Non-Functional Requirements

Area	Requirement
Code Quality	Code should be modular, readable, documented, testable, and free from hardcoded secrets.
Error Handling	Handle invalid questions, invalid SQL, missing database files, empty results, and connection failures.
Maintainability	Separate database logic, tool logic, prompt logic, agent creation, and application entry point.
Security	Allow only read-only database operations and reject unsafe SQL statements.
Usability	Responses should be clear for business users and should not require SQL knowledge.

14. Optional Advanced Features

- Streamlit UI with a simple chat interface and example queries sidebar.

- Display generated SQL query for debugging or learning purposes.
- Log intermediate steps such as user question, selected tool, generated SQL, raw database result, and final answer.
- Add LangSmith tracing or an equivalent observability mechanism.
- Add automated tests for at least five business questions.
- Add support for CSV export of tabular answers.

15. Expected Deliverables

42. Source code.
43. SQLite database creation script.
44. Sample seeded database.
45. LangChain legacy implementation, if attempted.
46. LangChain modern implementation, if attempted.
47. README.md with setup instructions.
48. sample_queries.md with at least 15 tested questions.
49. Screenshots or logs of successful execution.
50. Short explanation of tool design and safety checks.

16. README Requirements

- Project overview.
- Business use case.
- Technology stack.
- Database schema.
- Setup instructions.
- How to create and seed the database.
- How to run the legacy LangChain version.
- How to run the modern LangChain version.
- Environment variables required.
- Sample questions.
- Known limitations.
- Future improvements.

17. Evaluation Rubric

Criteria	Marks
SQLite database schema and sample data	15
Correct custom database tool implementation	20
SQL safety validations	15
LangChain agent implementation	20
Natural language query handling	10
Business-friendly responses	10
Code structure and readability	5
README and documentation	5
Total	100

Additional evaluator checks should include whether the agent uses the tool instead of hallucinating answers, whether JOIN-based and aggregation queries work, whether unsafe SQL is blocked, and whether empty results are handled properly.

18. Example Expected Interactions

Example 1: Revenue Query

User: What is the total revenue from completed orders?

Possible SQL:

```
SELECT SUM(total_amount) AS total_revenue
FROM orders
WHERE status = 'completed';
```

Expected response:

The total revenue from completed orders is Rs. 1,25,400.

Example 2: Low Stock Query

User: Which products are running low on stock?

Possible SQL:

```
SELECT name, category, stock_quantity
FROM products
WHERE stock_quantity < 10
ORDER BY stock_quantity ASC;
```

Expected response:

The following products are running low on stock:

1. Wireless Mouse - Electronics - 4 units
2. Yoga Mat - Fitness - 6 units
3. Coffee Mug - Kitchen - 8 units

Example 3: Top Customer Query

User: Which customer has spent the most money?

Possible SQL:

```
SELECT c.name, SUM(o.total_amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'completed'
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC
LIMIT 1;
```

Expected response:

The highest spending customer is Priya Sharma, with a total completed purchase value of Rs. 38,900.

19. Final Submission Checklist

- ☐ ecommerce.db is created successfully

- ☐ Tables are created correctly
- ☐ Sample data is inserted
- ☐ Database tool connects to SQLite
- ☐ Tool allows only SELECT queries
- ☐ Tool blocks unsafe SQL
- ☐ Agent can call the tool
- ☐ Agent answers at least 15 sample queries
- ☐ README is complete
- ☐ Code runs from a clean environment
- ☐ No secrets are committed
- ☐ Both LangChain version approaches are clearly documented, if attempted

20. Final Notes for Participants

Do not hardcode answers. The agent must retrieve answers from the database. The database schema should be included in the agent prompt so that the agent understands available tables and columns. The agent must not modify the database. The final response should be clear, concise, and useful for a business user.

This assignment gives practical exposure to agent building, tool integration, database interaction, SQL reasoning, version migration awareness, and safety controls that are important for production-grade AI assistant development.